

CSA0614-TOPIC 6-DAA

Y Greeshma Reddy

192424146

SIMATS Online Lab

greeshmareddyyanamala-alt/C... x +

Not secure simatslab.saveetha.com/practice_app/classactivity/attempt?assignment_id=1232

Ask Gemini

SIMATS ENGINEERING SIMATS Online Programming Lab DAA PROGRAMMING YANMALA GREESHMA REDDY

LAB PROGRAM - TOPIC 6

Language Python C++ Java

QUESTIONS

Q1 Merge Sort with Comparison Count 10 marks

Q2 Quick Sort with Comparison Count 10 marks

Q3 Merge Sort with Quick Sort Comparison Count 10 marks

Q4 Merge Sort Recursion Tree Visualization 10 marks

Q5 Quick Sort Worst Case Analysis of Quick Sort 10 marks

Q6 Quick Sort with Quick Sort Comparison Count 10 marks

Q7 Quick Sort Complexity Using Merge Sort 10 marks

Q8 Quick Sort Recursion Depth Calculation 10 marks

Q9 Merge Sort Process Visualization 10 marks

Q10 Space Complexity Comparison Merge Sort Quick Sort 10 marks

10/10 attempted

Merge Sort with Comparison Count 10 marks

Implement the Merge Sort algorithm to sort a given array of integers. The program should recursively divide the array into smaller subarrays and then merge them in a sorted manner. While merging, count the number of element comparisons performed. Ensure that the comparison counter is incremented at every comparison step. Finally, print the sorted array along with the total number of comparisons made during execution.

Test Cases:

Input: N = 8, A[] = {12, 4, 78, 23, 45, 67, 89, 1}

Output: 1, 4, 12, 23, 45, 67, 78, 89

Comparisons: 17

Input: N = 6, A[] = {5, 2, 8, 1, 6, 3}

Output: 1, 2, 3, 5, 6, 8

Comparisons: 11

Your Code

```
def merge_sort(arr):
    comparisons = 0
    def merge_sort_rec(arr):
        n = len(arr)
        if n <= 1:
            return arr
        mid = n // 2
        left = merge_sort_rec(arr[:mid])
        right = merge_sort_rec(arr[mid:])
        merge(left, right)
        return merge
    def merge(left, right):
        merged = []
        i = j = k = 0
        while i < len(left) and j < len(right):
            comparisons += 1
            if left[i] <= right[j]:
                merged.append(left[i])
                i += 1
            else:
                merged.append(right[j])
                j += 1
        while i < len(left):
            merged.append(left[i])
            i += 1
        while j < len(right):
            merged.append(right[j])
            j += 1
        return merged
    return merge_sort_rec(arr)

arr = [12, 4, 78, 23, 45, 67, 89, 1]
sorted_arr = merge_sort(arr)
print(sorted_arr, comparisons)
```

Input

8
1 2 3 4 5 6 7 8

Output

Sorted Array: 1, 2, 3, 4, 5, 6, 7, 8
Comparisons: 17

SIMATS Online Lab

greeshmareddyyanamala-alt/C... x +

Not secure simatslab.saveetha.com/practice_app/classactivity/attempt?assignment_id=1232

Ask Gemini

SIMATS ENGINEERING SIMATS Online Programming Lab DAA PROGRAMMING YANMALA GREESHMA REDDY

LAB PROGRAM - TOPIC 6

Language Python C++ Java

QUESTIONS

Q1 Merge Sort with Comparison Count 10 marks

Q2 Quick Sort with Comparison Count 10 marks

Q3 Merge Sort with Quick Sort Comparison Count 10 marks

Q4 Merge Sort Recursion Tree Visualization 10 marks

Q5 Quick Sort Worst Case Analysis of Quick Sort 10 marks

Q6 Quick Sort with Quick Sort Comparison Count 10 marks

Q7 Quick Sort Complexity Using Merge Sort 10 marks

Q8 Quick Sort Recursion Depth Calculation 10 marks

Q9 Merge Sort Process Visualization 10 marks

Q10 Space Complexity Comparison Merge Sort Quick Sort 10 marks

10/10 attempted

Quick Sort with Comparison Count 10 marks

Write a program to implement the Quick Sort algorithm using the last element as the pivot. The program should partition the array based on the pivot and recursively sort the subarrays. Count the number of comparisons made during the partitioning process and recursive calls. Ensure that the count is accurate for all cases. Display the sorted array along with the total number of comparisons.

Test Cases:

Input: N = 7, A[] = {38, 27, 43, 3, 8, 82, 10}

Output: 3, 8, 10, 27, 38, 43, 82

Comparisons: 15

Input: N = 5, A[] = {10, 7, 8, 9, 1}

Output: 1, 7, 8, 9, 10

Comparisons: 10

Your Code

```
def partition(arr, low, high):
    pivot = arr[high]
    i = low - 1
    comparisons = 0
    for j in range(low, high):
        comparisons += 1
        if arr[j] <= pivot:
            i += 1
            arr[j], arr[i] = arr[i], arr[j]
    arr[i], arr[high] = arr[high], arr[i]
    comparisons += 1
    return i
def quicksort(arr, low, high):
    total_comparisons = 0
    if low < high:
        p = partition(arr, low, high)
        total_comparisons += comp
        left_comp = quicksort(arr, low, p - 1)
        right_comp = quicksort(arr, p + 1, high)
        total_comparisons += left_comp + right_comp
    return total_comparisons
def merge():
    arr = [38, 27, 43, 3, 8, 82, 10]
    arr = list(map(int, input().split()))
    n = len(arr)
    comp = quicksort(arr, 0, n - 1)
    print(arr, comp)
```

Input

7
3 8 10 27 38 43 82

Output

3 8 10 27 38 43 82
Comparisons: 15

SIMATS Online Lab

greeshmareddyyanamala-alt/C... | +

Ask Gemini

Not secure simatslab.saveetha.com/practice_app/classactivity/attempt?assignment_id=1232

SIMATS ENGINEERING

SIMATS Online Programming Lab

DATA PROGRAMMING

YANMALA GREESHMA REDDY

LAB PROGRAM - TOPIC 6

Language Python C++ Java

QUESTIONS

Q1 Merge Sort with Comparison 10 marks

Q2 Merge Sort with Comparison 10 marks

Q3 Merge Sort with Quick Sort Comparison 10 marks

Q4 Merge Sort with Quick Sort Comparison 10 marks

Q5 Merge Sort with Quick Sort Comparison 10 marks

Q6 Merge Sort with Quick Sort Comparison 10 marks

Q7 Merge Sort with Quick Sort Comparison 10 marks

Q8 Merge Sort with Quick Sort Comparison 10 marks

Q9 Merge Sort with Quick Sort Comparison 10 marks

Q10 Merge Sort with Quick Sort Comparison 10 marks

10/10 answered

Merge Sort in Quick Sort Comparison 10 marks

Develop a program that implements both Merge Sort and Quick Sort in a single program. The same input array should be used for both algorithms to ensure fairness in comparison. Count the number of comparisons separately for each algorithm. After sorting, display the sorted array along with the comparison counts of both methods. Analyze which algorithm performs better for the given input.

Test Cases:

Input N = 6, A[] = {2, 1, 1, 3, 5, 7}

Output Sorted Array: 1, 1, 2, 3, 5, 7 Merge Comparisons: 11 Quick Comparisons: 12

Input N = 5, A[] = {4, 2, 6, 1, 3}

Output Sorted Array: 1, 2, 3, 4, 6 Merge Comparisons: 8 Quick Comparisons: 9

Your Code

```
def merge_sort_count(arr):
    comparisons = 0
    def merge_sort(arr):
        nonlocal comparisons
        if len(arr) > 1:
            mid = len(arr) // 2
            left = arr[:mid]
            right = arr[mid:]
            merge_sort(left)
            merge_sort(right)
            merge(left, right)
        comparisons += 1
    def merge(left, right):
        nonlocal comparisons
        i = j = k = 0
        while i < len(left) and j < len(right):
            comparisons += 1
            if left[i] <= right[j]:
                arr[k] = left[i]
                i += 1
            else:
                arr[k] = right[j]
                j += 1
            k += 1
        while i < len(left):
            arr[k] = left[i]
            i += 1
            k += 1
        while j < len(right):
            arr[k] = right[j]
            j += 1
            k += 1
    merge_sort(arr)
    return arr, comparisons

def quick_sort_count(arr):
    comparisons = 0
    def quick_sort(arr, low, high):
        nonlocal comparisons
        if low < high:
            pivot = arr[high]
            i = low - 1
            for j in range(low, high):
                comparisons += 1
                if arr[j] < pivot:
                    i += 1
                    arr[i], arr[j] = arr[j], arr[i]
            arr[i+1], arr[high] = arr[high], arr[i+1]
            quick_sort(arr, low, i)
            quick_sort(arr, i+1, high)
    quick_sort(arr, 0, len(arr)-1)
    return arr, comparisons
```

Input

5
2 1 1 3 5 7

Output

Sorted Array : 1, 1, 2, 3, 5, 7
Merge Comparisons : 11
Quick Comparisons : 12
Merge Sort performs better for this input.

SIMATS Online Lab

greeshmareddyyanamala-alt/C... | +

Ask Gemini

Not secure simatslab.saveetha.com/practice_app/classactivity/attempt?assignment_id=1232

SIMATS ENGINEERING

SIMATS Online Programming Lab

DATA PROGRAMMING

YANMALA GREESHMA REDDY

LAB PROGRAM - TOPIC 6

Language Python C++ Java

QUESTIONS

Q1 Merge Sort with Comparison 10 marks

Q2 Merge Sort with Comparison 10 marks

Q3 Merge Sort with Quick Sort Comparison 10 marks

Q4 Merge Sort with Quick Sort Comparison 10 marks

Q5 Merge Sort with Quick Sort Comparison 10 marks

Q6 Merge Sort with Quick Sort Comparison 10 marks

Q7 Merge Sort with Quick Sort Comparison 10 marks

Q8 Merge Sort with Quick Sort Comparison 10 marks

Q9 Merge Sort with Quick Sort Comparison 10 marks

Q10 Merge Sort with Quick Sort Comparison 10 marks

10/10 answered

Merge Sort Execution Time Measurement 10 marks

Write a program to measure the execution time of the Merge Sort algorithm. Use appropriate timing functions to record the start and end time of execution. Apply the algorithm on different types of inputs such as sorted and reverse sorted arrays. Display the sorted output along with the time taken for execution. Analyze how input order affects performance.

Test Cases:

Input N = 7, A[] = {5, 7, 6, 5, 4, 3, 2}

Output 3, 2, 1, 4, 5, 6, 7

Time Taken: ~0.0002 sec

Input N = 5, A[] = {2, 1, 3, 4, 5}

Output 1, 2, 3, 4, 5

Time Taken: ~0.0002 sec

Your Code

```
import time

def merge_sort(arr):
    if len(arr) > 1:
        mid = len(arr) // 2
        left = arr[:mid]
        right = arr[mid:]
        merge_sort(left)
        merge_sort(right)
        merge(left, right)
    def merge(left, right):
        i = j = k = 0
        while i < len(left) and j < len(right):
            if left[i] <= right[j]:
                arr[k] = left[i]
                i += 1
            else:
                arr[k] = right[j]
                j += 1
            k += 1
        while i < len(left):
            arr[k] = left[i]
            i += 1
            k += 1
        while j < len(right):
            arr[k] = right[j]
            j += 1
            k += 1
    merge_sort(arr)
    return arr

def measure_execution_time(arr):
    start = time.perf_counter()
    sorted_arr = merge_sort(arr)
    end = time.perf_counter()
    return sorted_arr, end - start
```

Input

5
2 1 3 4 5

Output

Time Taken : ~0.0002 sec

SIMATS Online Lab

greeshmareddyyanamala-alt/C... x +

Ask Gemini

simatslab.saveetha.com/practice_app/classactivity/attempt?assignment_id=1232

SIMATS ENGINEERING

SIMATS Online Programming Lab

DATA PROGRAMMING

YANMALA GREESHMA REDDY

LAB PROGRAM - TOPIC 6

Language Python C++ Java

QUESTIONS

Q1 Merge Sort with Comparison Count

Q2 Quick Sort with Comparison Count

Q3 Merge Sort with Quick Sort Comparison

Q4 Merge Sort Recursive Function

Q5 Quick Sort Worst Case Analysis of Quick Sort

Q6 Merge Sort with Merge Sort

Q7 Counting Inversions using Merge Sort

Q8 Quick Sort Recursive Depth

Q9 Extended Merge Process

Q10 Space Complexity Comparison Merge vs Quick Sort

10/10 answered

Counting Inversions using Merge Sort

10 marks

Develop a program to count the number of inversions in an array using Merge Sort. An inversion is defined as a pair (i, j) such that (i < j) and (A[i] > A[j]). Modify the merge step to count such inversions efficiently. Print both the sorted array and the inversion count. This helps visualize how for the array is from being sorted.

Test Cases:

Input: N = 5, A[] = [5,4,1,3,2]
Output Sorted: [1,2,3,4,5] Inversions: 7
Input: N = 4, A[] = [4,3,2,1]
Output Sorted: [1,2,3,4] Inversions: 6

Your Code

```
def merge_sort_count_inver(arr):
    if len(arr) <= 1:
        return arr, 0
    mid = len(arr) // 2
    left = arr[:mid]
    right = arr[mid:]
    a, p = merge_sort_count_inver(left)
    b, q = merge_sort_count_inver(right)
    merged, r = merge(left, right)
    inv_count = p + q + r
    return merged, inv_count

def merge(arr1, arr2):
    merged = []
    i = j = 0
    while i < len(arr1) and j < len(arr2):
        if arr1[i] <= arr2[j]:
            merged.append(arr1[i])
            i += 1
        else:
            merged.append(arr2[j])
            j += 1
    merged.extend(arr1[i:])
    merged.extend(arr2[j:])
    return merged

# Test Cases
arr = [5,4,1,3,2]
sorted_arr, inv_count = merge_sort_count_inver(arr)
print(sorted_arr)
print(inv_count)
```

Input

5
5,4,1,3,2

Output

Sorted: [1,2,3,4,5]
Inversions: 7

29°C Sunny

Search

ENG IN

08:50 22-08-2026

SIMATS Online Lab

greeshmareddyyanamala-alt/C... x +

Ask Gemini

simatslab.saveetha.com/practice_app/classactivity/attempt?assignment_id=1232

SIMATS ENGINEERING

SIMATS Online Programming Lab

DATA PROGRAMMING

YANMALA GREESHMA REDDY

LAB PROGRAM - TOPIC 6

Language Python C++ Java

QUESTIONS

Q1 Merge Sort with Comparison Count

Q2 Quick Sort with Comparison Count

Q3 Merge Sort with Quick Sort Comparison

Q4 Merge Sort Recursive Function

Q5 Quick Sort Worst Case Analysis of Quick Sort

Q6 Merge Sort with Merge Sort

Q7 Counting Inversions using Merge Sort

Q8 Quick Sort Recursive Depth

Q9 Extended Merge Process

Q10 Space Complexity Comparison Merge vs Quick Sort

10/10 answered

Quick Sort Recursive Depth Analysis

10 marks

Implement Quick Sort and track the recursion depth during execution. Maintain a variable to store the maximum depth reached. Print the sorted array along with the maximum recursion depth. This helps analyze space complexity. Compare depth for different inputs.

Test Cases:

Input: N = 8, A[] = [7,8,6,5,1]
Output: [1,5,6,7,8] Max Depth: 4
Input: N = 5, A[] = [7,2,3,4,5]
Output: [2,3,4,5,7] Max Depth: 5

Your Code

```
def quicksort(arr, low, high, depth, max_depth_ref):
    if low >= high:
        return
    if depth > max_depth_ref[0]:
        max_depth_ref[0] = depth
    pivot = arr[high]
    i = low - 1
    for j in range(low, high):
        if arr[j] <= pivot:
            i += 1
            arr[i], arr[j] = arr[j], arr[i]
    arr[i], arr[high] = arr[high], arr[i]
    quicksort(arr, low, i - 1, depth + 1, max_depth_ref)
    quicksort(arr, i + 1, high, depth + 1, max_depth_ref)

def quicksort_with_max_depth(arr):
    max_depth_ref = [0]
    n = len(arr)
    if n <= 1:
        return arr
    quicksort(arr, 0, n - 1, 0, max_depth_ref)
    return arr, max_depth_ref[0]

# Test Cases
arr = [7,8,6,5,1]
sorted_arr, max_depth = quicksort_with_max_depth(arr)
print(sorted_arr)
print(max_depth)
```

Input

8
7,8,6,5,1

Output

Sorted: [1,5,6,7,8]
Max Depth: 4

29°C Sunny

Search

ENG IN

08:50 22-08-2026

